CS-253 Design and Analysis of Algorithms

Practical Assignment: Study of Sorting Algorithm Performance

P Mokshagna (241CS244)
R Shiva Kumar (241CS245)

April 10, 2026

1 Introduction

Sorting is a basic yet essential operation in computer science, widely used to improve the efficiency of many computational tasks. While there are many sorting techniques available, their efficiency differs depending on factors like input size and the arrangement of data.

This assignment focuses on comparing the practical behavior of different sorting algorithms and linking those observations with their theoretical complexities. The algorithms included in this study are Selection Sort, Bubble Sort, Insertion Sort, Merge Sort, Quick Sort (with different pivot strategies), Heap Sort, and Radix Sort.

Instead of only implementing these algorithms, the primary goal is to observe how they behave under different types of input. For this purpose, three input cases are considered: random data, already sorted data, and reverse sorted data. These represent average, best, and worst-case scenarios respectively.

Additionally, different pivot selection methods in Quick Sort are analyzed to understand how pivot choice impacts performance. The study also explores how execution time relates to the number of comparisons.

Through systematic experimentation, this report aims to present a clear understanding of how different sorting algorithms perform in real-world scenarios.

2 Data Generation and Experimental Setup

To ensure accurate and consistent evaluation of the sorting algorithms, a structured experimental setup was used. The details of the system and methodology are described below

- **Machine Specifications:** All experiments were conducted on the following system:
  - Processor: 12th Gen Intel Core i5-1235U (1.30 GHz)
  - RAM: 16 GB

  - System Type: 64-bit OS, x64-based processor

- **Timing Method:** Execution time was measured within the C++ code using `std::chrono::high_re` and recorded in microseconds.

- **Input Sizes:** The following sizes were tested:

$$1000,\ 2000,\ 3000,\ 5000,\ 7000,\ 10000,\ 12000,\ 15000$$

- **Input Types:** For each size, three types of inputs were used:
  - Random data (average case)
  - Sorted data (best case for some algorithms)
  - Reverse sorted data (worst case)

- **Data Creation:** All inputs were generated within the program. Random arrays used pseudo-random generation, while sorted and reverse arrays were created directly.

- **Repetitions:** Each experiment was executed 5 times.

- **Final Measurement:** The average of the 5 runs was taken as the final execution time.

- **Consistency:** For each input type and size, the same base array was reused across runs to ensure fairness.

3 Which of the three versions of Quick Sort performs the best?

This section compares three Quick Sort variations based on pivot selection:

- First element as pivot

- Random element as pivot

- Median-of-three (first, middle, last)

The pivot plays a crucial role in dividing the array. Balanced partitions lead to efficient performance, while uneven splits degrade efficiency.

3.1 Theoretical Analysis

Quick Sort follows a divide-and-conquer approach, and its efficiency depends on how balanced the partitions are.

- **Best Case:** Occurs when partitions are evenly split. This results in logarithmic recursion depth and overall complexity of $O(n \log n)$.
- **Worst Case:** Happens when partitions are highly unbalanced (size 0 and $n-1$). This leads to $O(n^2)$ complexity.
- **Average Case:** With random data, partitions are usually balanced enough, giving expected performance of $O(n \log n)$.

Different pivot strategies aim to reduce the chances of worst-case behavior.

3.2 Empirical Results

3.2.1 Best Case Performance

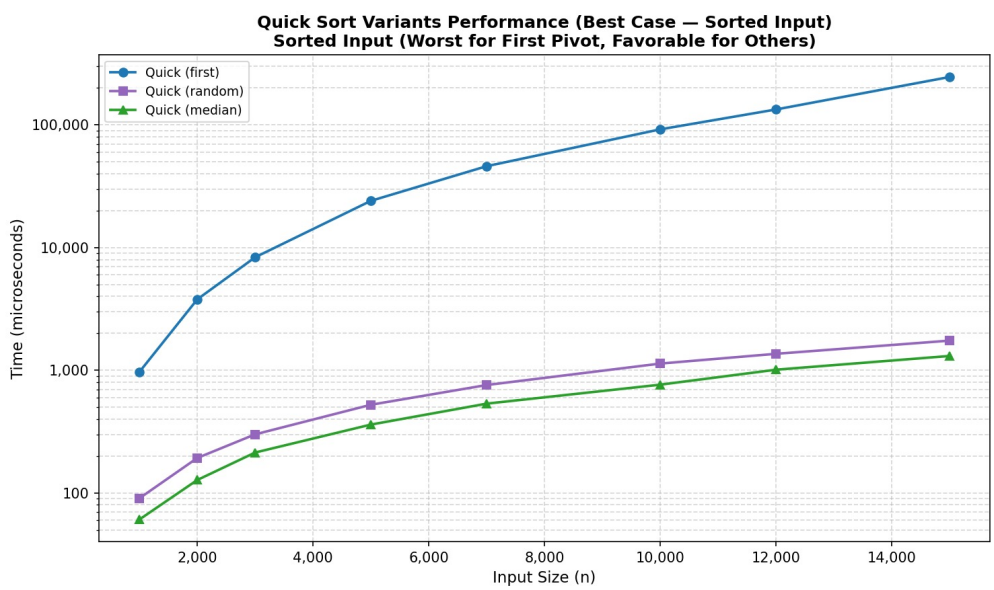

Figure 1: Quick Sort Variants on Sorted Input

The First Pivot method performs poorly on sorted input, as it always picks the smallest element, causing highly unbalanced partitions.

Random Pivot performs better by avoiding fixed patterns, while Median-of-Three gives the most balanced splits and best performance.

3.2.2 Worst Case Performance

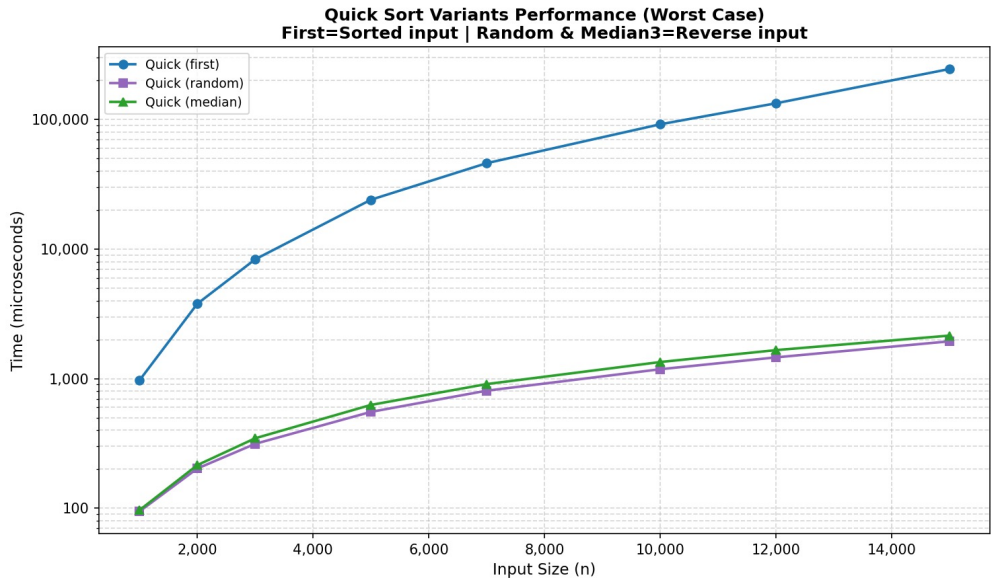

Figure 2: Quick Sort Variants Worst Case

First Pivot shows clear quadratic growth. Random and Median-of-Three avoid extreme imbalance, maintaining near $O(n \log n)$ behavior.

3.2.3 Average Case Performance

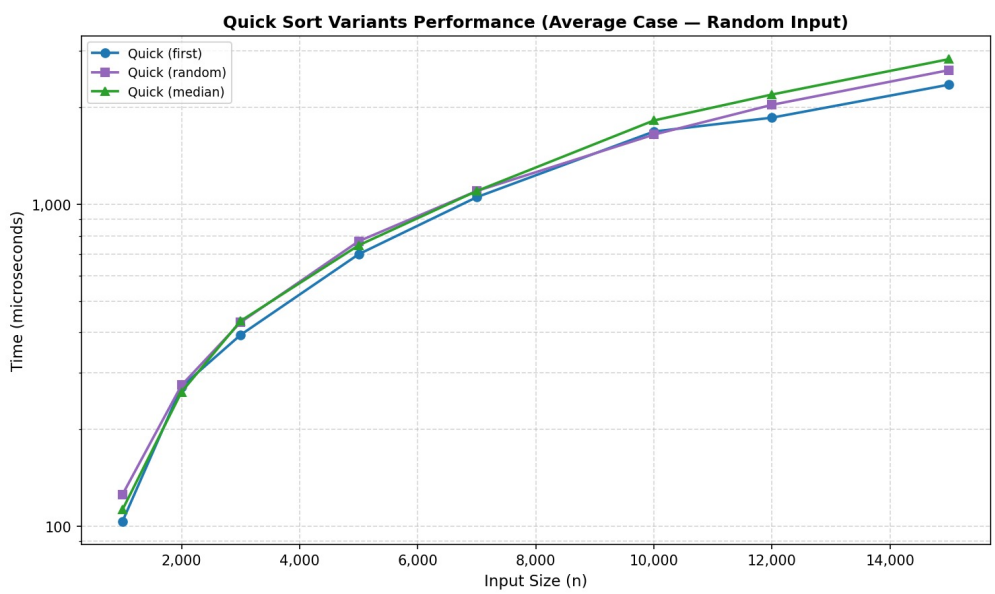

Figure 3: Quick Sort Variants Average Case

All methods perform close to $O(n \log n)$, but Random and Median-of-Three show more stable results.

3.3 Conclusion

Median-of-Three performs the best overall due to consistent partitioning. Random Pivot is also effective, while First Pivot is unreliable and sensitive to input order.

4 Which of the seven sorts performs the best?

This section compares seven sorting algorithms:

- Selection Sort
- Bubble Sort
- Insertion Sort
- Merge Sort
- Quick Sort (Median-of-Three)
- Heap Sort
- Radix Sort

4.1 Theoretical Complexities

Algorithm	Best	Average	Worst	Space
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)
Selection Sort	O(n^2)	O(n^2)	O(n^2)	O(1)
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick Sort (Med3)	O(n log n)	O(n log n)	O(n log n)	O(log n)
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)
Radix Sort	O(d(n+k))	O(d(n+k))	O(d(n+k))	O(n+k)

Table 1: Complexity Comparison

4.2 Empirical Analysis

4.2.1 Best Case

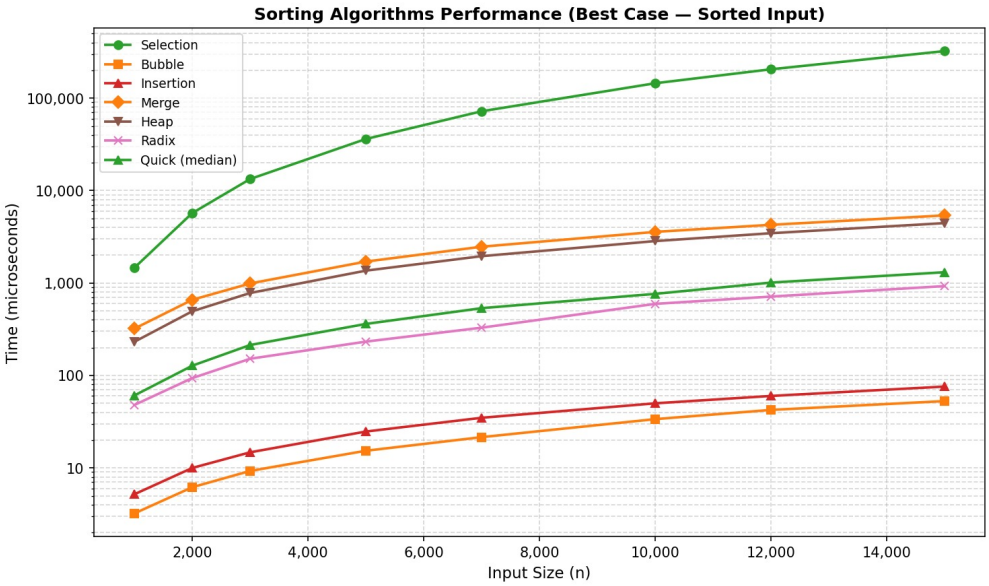

Figure 4: Best Case Performance

Bubble and Insertion Sort perform very well on sorted data due to their adaptive nature. Selection Sort does not benefit from input order.

Merge, Heap, and Quick Sort remain consistent, while Radix Sort performs efficiently due to its non-comparative nature.

4.2.2 Worst Case

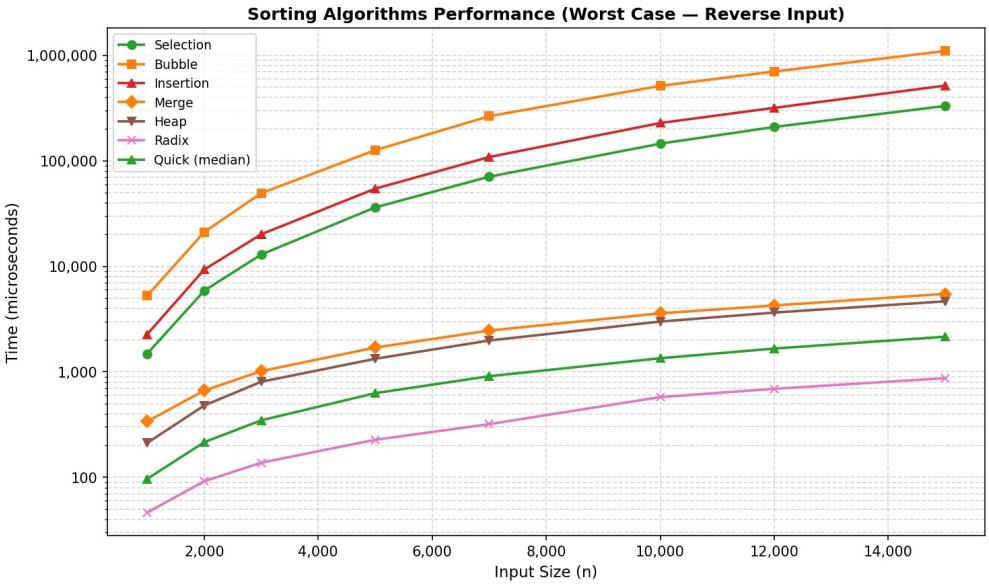

Figure 5: Worst Case Performance

Quadratic algorithms perform poorly. Merge, Heap, and Quick Sort remain stable. Radix Sort performs best again.

4.2.3 Average Case

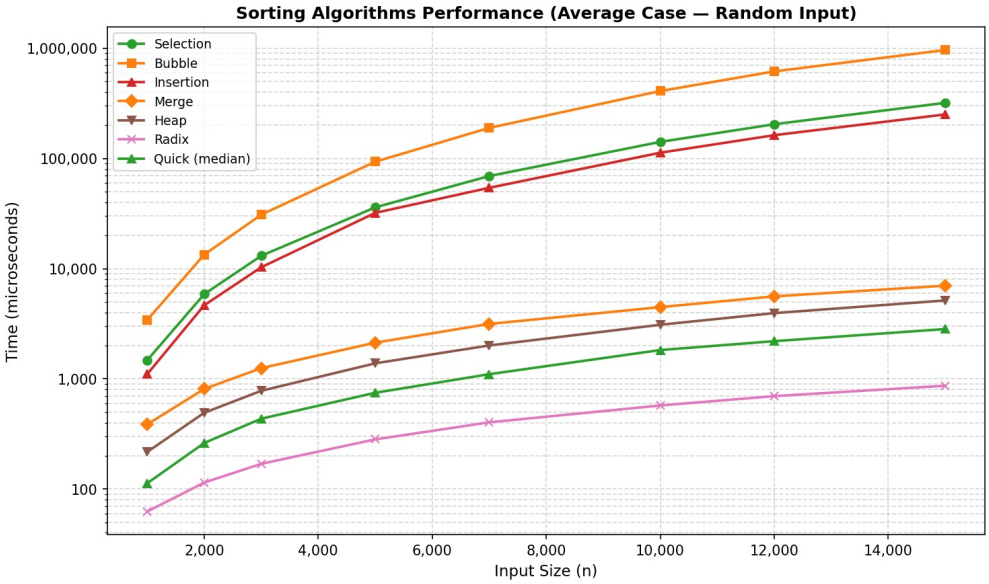

Figure 6: Average Case Performance

Efficient algorithms scale well, while quadratic ones slow down significantly. Radix Sort gives the best performance overall but has limitations.

4.3 Conclusion

Radix Sort performs best overall for large inputs. Among comparison-based methods, Quick Sort (Median-of-Three) is the most efficient in practice.

5 Correlation Between Comparisons and Execution Time

Execution time in comparison-based algorithms largely depends on the number of comparisons performed.

5.1 Analysis

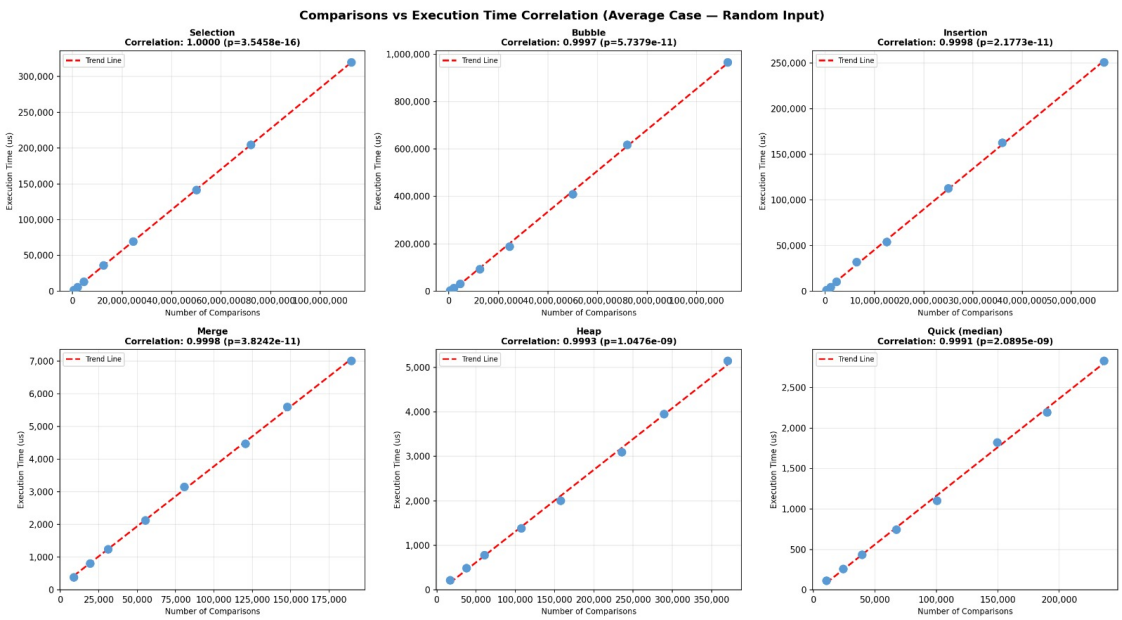

Figure 7: Comparisons vs Time

A strong positive correlation is observed. Algorithms with more comparisons take more time.

5.2 Conclusion

Reducing comparisons directly improves efficiency, especially for large inputs.

6 Final Conclusion

This study provided a comprehensive comparison of different sorting algorithms under various input conditions such as random, sorted, and reverse-sorted data. The experimental observations closely matched the expected theoretical time complexities, indicating that asymptotic analysis is a reliable way to estimate algorithm performance.

From the evaluation of Quick Sort variants, it became evident that pivot selection plays a critical role in determining performance. The Median-of-Three strategy performed the best overall, as it consistently produced balanced partitions and effectively avoided worst-case scenarios. The Random Pivot approach also showed good performance, while the First Pivot method was highly sensitive to input order and often resulted in inefficient behavior.

When comparing all seven sorting algorithms, a clear distinction was observed between quadratic-time algorithms and those with logarithmic growth. Selection Sort, Bubble Sort, and Insertion Sort exhibited a rapid increase in execution time as input size increased, making them unsuitable for large datasets. However, they performed efficiently for smaller inputs or nearly sorted data.

In contrast, Merge Sort, Heap Sort, and Quick Sort demonstrated stable and efficient performance across all input types due to their $O(n \log n)$ complexity. Among these, Quick Sort with the Median-of-Three strategy delivered the best practical performance in most cases.

Radix Sort outperformed all comparison-based algorithms for large datasets since it does not rely on comparisons and achieves near-linear time complexity. However, its effectiveness depends on the nature and structure of the input data.

The correlation analysis revealed a strong positive relationship between the number of comparisons and execution time in comparison-based algorithms. This confirms that minimizing comparisons is an important factor in improving efficiency.

Overall, the findings indicate that no single sorting algorithm is optimal for every situation. The choice of algorithm should depend on factors such as input size, data distribution, and specific application requirements.

The graphical results further support these conclusions, as the observed growth patterns closely align with the expected theoretical complexities. This emphasizes the importance of combining theoretical understanding with experimental validation when analyzing algorithm performance.